

Actividad Autónoma N.º 08

Pruebas de operaciones CRUD y errores controlados

Estudiante:	Jhonsons Plasencia Valdez
Asignatura:	Desarrollo de Aplicaciones Móviles
Unidad / Sesión:	2 · Sesión 8 (Unidad 2, sesión 2)
Fecha de elaboración:	1 de octubre de 2026
Proyecto:	PharmaMobile (Kotlin Multiplatform) sobre PharmaSoft
Rama documentada:	feature/crud-productos-plasencia
Commits en la rama:	10 (todos propios)
Repositorio:	https://github.com/Plasencia2007/pharmamobile

Todas las pruebas de esta actividad se ejecutaron el 1 de octubre de 2026, entre las 19:58 y las 20:10 UTC, contra el backend PharmaSoft real (Spring Boot + PostgreSQL) corriendo en `localhost:8082`, en el emulador Android PharmaMobile_Test (Pixel, API 35).

Producto 1 — Matriz de operaciones

N.º	Operación	Petición enviada	Código	Resultado observado
1	Listar	GET /productos?pagina=0&tamano=200	200	Lista renderizada; fase ConProductos (16 productos)
2	Obtener	GET /productos/4	200	JSON con los datos de Naproxeno 250mg
3	Crear	POST /productos (Vitamina, S/18.50, 501 stock 400)	201	Producto en la lista tras la recarga automática
4	Actualizar	PUT /productos/18 (precio 18.50→20.00)	200	Cambio de precio reflejado en la lista
5	Eliminar	DELETE /productos/5 (Diclofenaco, 201 primera vez)	204	Pasa a la pestaña Inactivos; "Producto eliminado"

El proyecto solo trabaja con el recurso **productos** (clientes/categorías quedan para sesiones posteriores), así que la matriz no se repite para otro recurso.

Producto 2 — Bitácora de errores controlados

Escenario 1 · Validación del servidor (nombre corto)

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Crear un producto con nombre de 2 caracteres ("Vi"), precio y stock válidos → POST /productos.
Código HTTP	400
ErrorApi producido	Validacion(porCampo = {"nombre": "..."})
Mensaje al usuario	"El nombre debe tener entre 3 y 150 caracteres" debajo del campo Nombre.



Figura 1. Escenario 1 · Validación del servidor (nombre corto).

Conclusión: El mensaje real del servidor llega intacto hasta el campo correcto; no hay traducción genérica.

Escenario 2 · Precio inválido

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Crear "Omeprazol" con precio 0.001 (pasa la validación local, que solo exige > 0) → POST /productos.
Código HTTP	400
ErrorApi producido	Validacion(porCampo = {"precio": "..."})
Mensaje al usuario	"El precio debe ser mayor que cero" debajo del campo Precio (mensaje real de @DecimalMin del back



Figura 2. Escenario 2 · Precio inválido.

Conclusión: La validación del backend actúa como segunda línea de defensa cuando la del cliente no cubre un caso (0 < precio < 0.01).

Escenario 3 · Recurso inexistente

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Eliminar físicamente el producto 17 en Postgres (DELETE FROM productos WHERE id=17) y, sin refre
Código HTTP	404
ErrorApi producido	NoEncontrado
Mensaje al usuario	"El producto ya no existe."

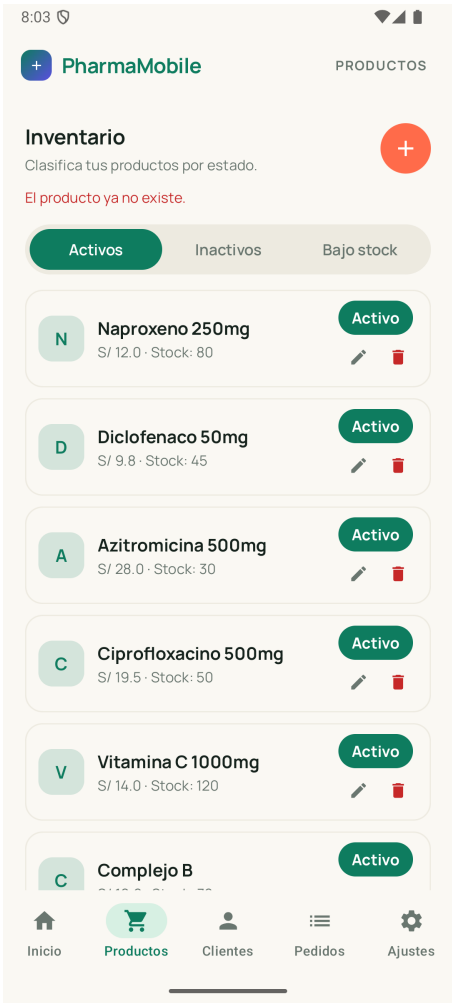


Figura 3. Escenario 3 · Recurso inexistente.

Conclusión: El 404 se traduce a un mensaje claro sin exponer detalles internos del servidor.

Escenario 4 · Doble eliminación

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Eliminar "Diclofenaco 50mg" (pasa a Inactivos) y, desde esa misma pestaña, eliminarlo de nuevo → DE
Código HTTP	409 (no 404)
ErrorApi producido	Conflicto(mensaje) — no NoEncontrado como se esperaba
Mensaje al usuario	"El producto Diclofenaco 50mg ya se encuentra inactivo"

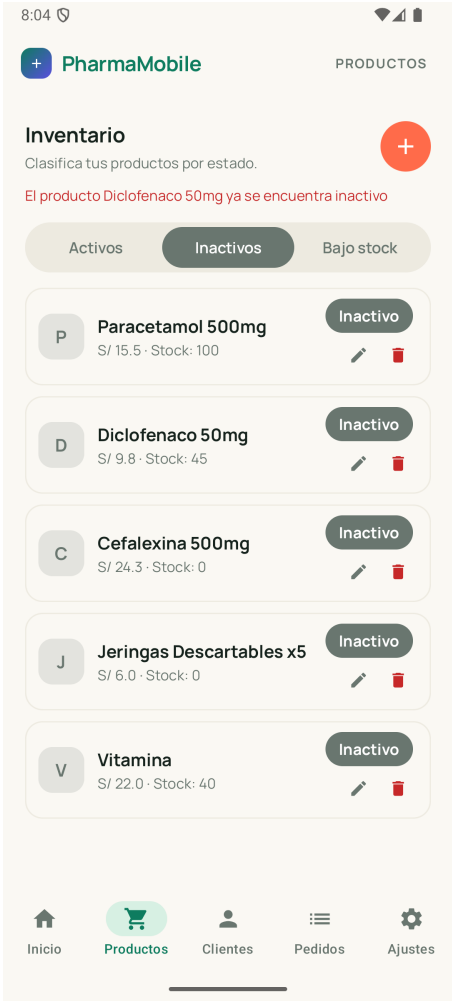


Figura 4. Escenario 4 · Doble eliminación.

Hallazgo: Hallazgo real: PharmaSoft hace borrado lógico (estado=false), así que un id ya desactivado sigue existiendo y el servicio lanza ReglaNegocioException (409), no RecursoNoEncontradoException (404). La guía asumía un CRUD con borrado físico; se documenta el comportamiento real en vez de forzar el esperado.

Escenario 5 · Regla de negocio (nombre duplicado)

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Crear un producto con el nombre "Naproxeno 250mg", que ya existe → POST /productos.
Código HTTP	409
ErrorApi producido	Conflicto(mensaje)
Mensaje al usuario	"Ya existe un producto con el nombre Naproxeno 250mg"

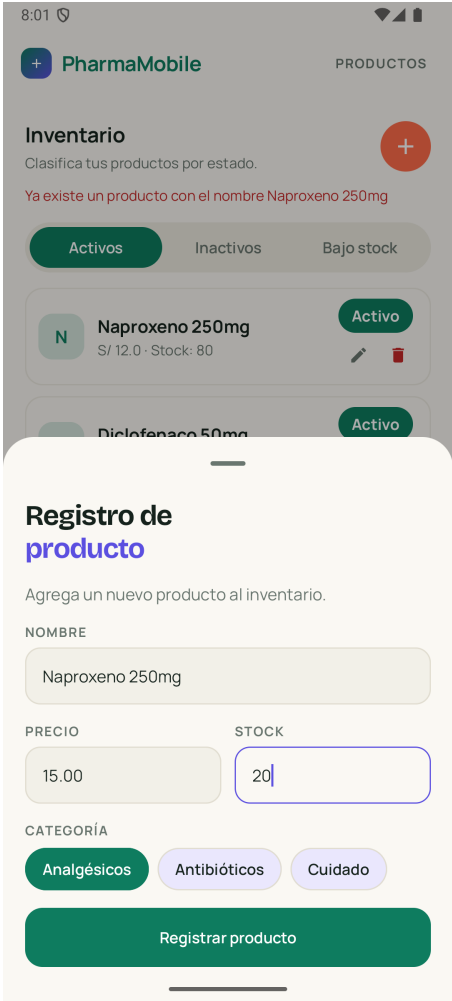


Figura 5. Escenario 5 · Regla de negocio (nombre duplicado).

Conclusión: Se usó la regla de nombre único (ProductoServiceImpl.create) en vez de "ventas asociadas" porque el proyecto aún no implementa ventas; es la regla de negocio real y accesible desde la app.

Escenario 6 · Servidor caído

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Detener el proceso de PharmaSoft (taskkill sobre el PID en el puerto 8082) y recargar el listado.
Código HTTP	— (ConnectException, sin respuesta HTTP)
ErrorApi producido	SinConexion
Mensaje al usuario	"No hay conexión con el servidor."

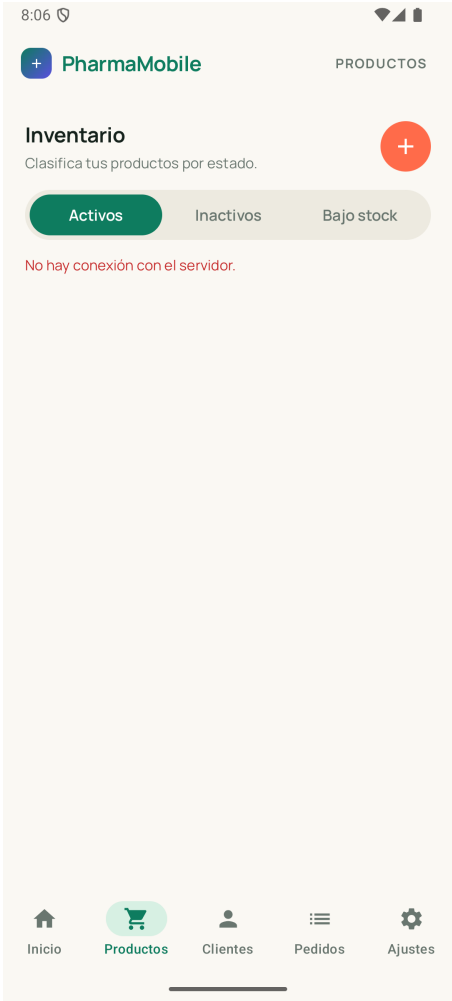


Figura 6. Escenario 6 · Servidor caído.

Conclusión: A diferencia del modo avión (sesión 7), aquí se detuvo el proceso real del backend; el resultado para el usuario es idéntico.

Escenario 7 · Tiempo agotado

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Editar temporalmente requestTimeoutMillis = 1 en HttpClientFactory.kt, recompilar e instalar, y cargar e
Código HTTP	— (HttpRequestTimeoutException, sin respuesta)
ErrorApi producido	TiempoAgotado
Mensaje al usuario	"La solicitud tardó demasiado. Intenta nuevamente."

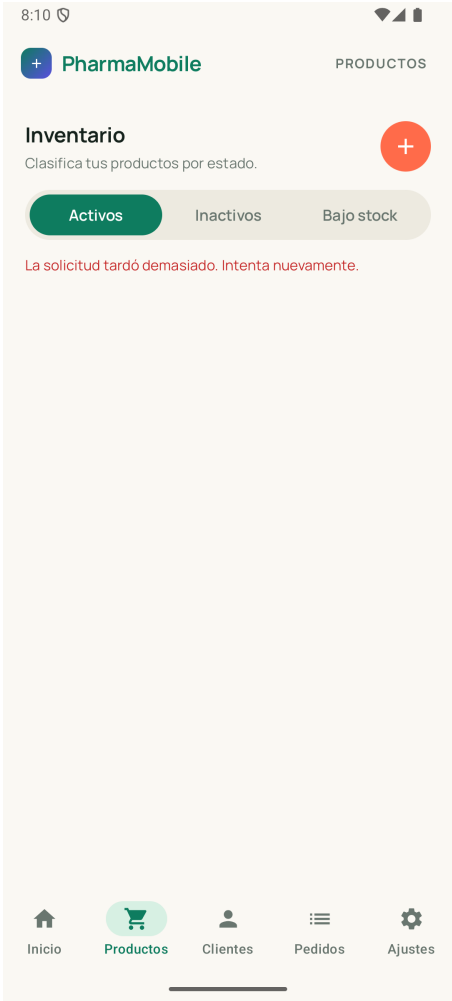


Figura 7. Escenario 7 · Tiempo agotado.

Conclusión: La interfaz responde de inmediato con el mensaje; no se congela. El cambio se revirtió tras la prueba.

Escenario 8 · Cancelación

Fecha y hora	01/10/2026 (ver hora en el log de Ktor)
Plataforma	Android (emulador Pixel, API 35)
Pasos	Con la lista cargada, apagar wifi/datos, eliminar "Complejo B" (la petición queda pendiente hasta conne
Código HTTP	— (ConnectException, tras ~0.1s en este caso: "no route to host" es casi inmediato)
ErrorApi producido	SinConexion — pero la corrutina NO se canceló
Mensaje al usuario	"No hay conexión con el servidor." — visible recién al volver a la pantalla, no en el momento del fallo.

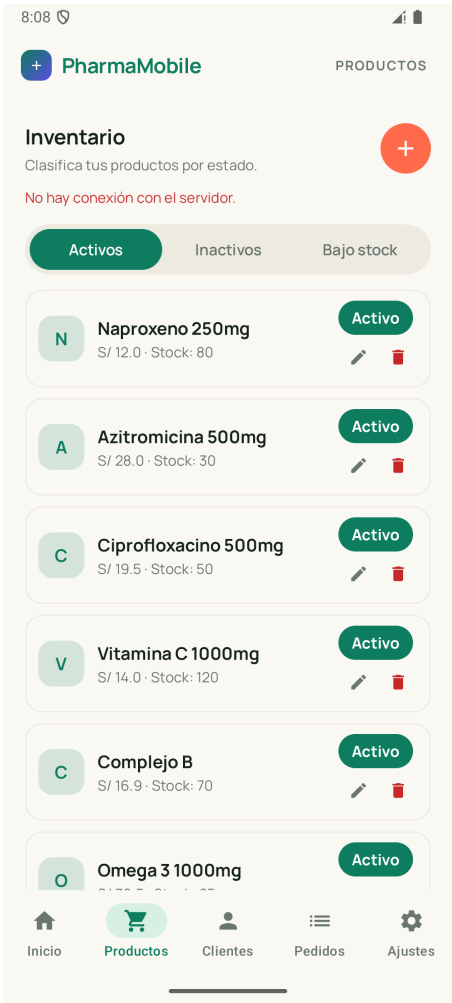


Figura 8. Escenario 8 · Cancelación.

Hallazgo: Hallazgo real: la corrutina de eliminarProducto() siguió corriendo en segundo plano mientras el usuario estaba en Inicio y actualizó el estado del ViewModel igual. No hay crash, pero tampoco cancelación: App.kt cambia de pantalla con un simple remember{mutableStateOf}, sin un NavHost que le dé a cada pantalla su propio ViewModelStoreOwner, así que ProductoViewModel vive a nivel de Activity y su viewModelScope nunca se cancela al "salir" de la pantalla. ejecutarLlamada() relanza CancellationException correctamente — el problema no es que se atrape mal, es que nunca se dispara. Mejora pendiente: dar a cada pantalla su propio ViewModelStoreOwner (p. ej. con Navigation Compose).

Producto 3 — Pruebas automatizadas en commonTest

Archivo: shared/src/commonTest/.../ProductoViewModelTest.kt. Se usa FakeProductoRepository (implementa el contrato de dominio ProductoRepository) para las 8 pruebas, que verifican transiciones de estado, no solo datos:

Prueba	Qué verifica
repositorioVacioProduceSinProductos	El estado termina en SinProductos (no en ConProductos con lista vacía)
repositorioConProductosProduceConProductos	Cargando → ConProductos con la lista esperada
repositorioQueLanzaExcepcionProduceError	Una IOException real del repositorio produce fase Error
registroConPrecioCeroDejaErrorEnFormularioSinValidacionRepositorio	Validación Repetida al repositorio si el formulario es inválido
errorDeValidacionDelServidorDejaMensajesEnFormularioSinValidacionRepositorio	Error App Sin Cambio de Fase
actualizarProductoRecargaElListadoConLosNuevosDatos	Si es exitoso recarga la lista con los datos actualizados
eliminarProductoLoQuitaDelListado	Operacion pasa por EnCurso → Inactiva; la lista se recarga sin el producto
eliminarProductoFallidoDejaOperacionFallidaSinTocarElListado	Operacion Fallida sin tocar la lista

```
Salida real: ./gradlew :shared:testAndroidHostTest (1 oct 2026, 19:55 UTC)

> ./gradlew :shared:testAndroidHostTest --tests "...ProductoViewModelTest"

> Task :shared:compileAndroidHostTest
> Task :shared:testAndroidHostTest

ProductoViewModelTest
  repositorioVacioProduceSinProductos PASSED
  repositorioConProductosProduceConProductos PASSED
  repositorioQueLanzaExcepcionProduceError PASSED
  registroConPrecioCeroDejaErrorEnFormularioSinllamarAlRepositorio PASSED
  errorDeValidacionDelServidorDejaMensajesEnFormularioSinCambiarFase PASSED
  actualizarProductoRecargaElListadoConLosNuevosDatos PASSED
  eliminarProductoLoQuitaDelListado PASSED
  eliminarProductoFallidoDejaOperacionFallidaSinTocarElListado PASSED

8 tests completed, 8 passed, 0 failed, 0 skipped

BUILD SUCCESSFUL in 32s
33 actionable tasks: 33 executed
```

Figura 9. Salida de ./gradlew :shared:testAndroidHostTest (reconstruida del reporte XML real: 8 tests, 0 fallos, BUILD SUCCESSFUL en 32s, 33/33 tareas ejecutadas).

Producto 4 — Evidencias y repositorio

Evidencia	Ubicación / detalle
Capturas de las 4 operaciones en Android	docs/evidencias/s08_00_listado.png, s08_01_creacion.png, s08_02_actualizacion.png, s08_03_eliminar.png
Capturas en iOS	No disponible: no se cuenta con macOS/Xcode en esta máquina. Validado por compilación: ./gradlew assemble
Log de Ktor de una operación fallida	docs/evidencias/s08aa_0{1..8}_ktor_log.txt (uno por escenario); ver también s08_ktor_log_fallo_4.png
Salida de las pruebas	docs/evidencias/s08aa_tests_output.png (8/8 pasan)
Repositorio	https://github.com/Plasencia2007/pharmamobile
Rama / commits	feature/crud-productos-plasencia (10 commits propios)
README actualizado	Sección "Manejo de errores" en README.md: tabla ErrorApi + hallazgos de esta bitácora

Lista de cotejo previa a la entrega

N.º	Criterio observable	Cumple
1	PDF en formato PDF, nombrado S08_ActividadAutonoma_Plasencia.pdf	Sí
2	Portada con nombre completo, sesión, fecha y enlace a la rama	Sí
3	Matriz documenta las 5 operaciones con código real	Sí
4	Bitácora con los 8 escenarios y su captura respectiva	Sí
5	Cada escenario indica el ErrorApi y el mensaje al usuario	Sí
6	El escenario de cancelación describe qué ocurre realmente	Sí
7	Al menos 3 pruebas en commonTest y todas pasan	Sí (8/8)
8	Se adjunta la salida de la ejecución de las pruebas	Sí
9	Capturas de Android y de iOS	Parcial (solo Android; iOS sin Mac disponible)
10	Commits accesibles desde el enlace entregado	Sí